Data Engineering Pipeline Manual

Architecture, Ingestion, Transformation & Quality Framework | Version 2.0

1. Pipeline Architecture Overview

TechViet's data engineering pipeline is responsible for collecting, transforming, and loading operational data from the ChatAssist platform and internal systems into the analytical data warehouse for business intelligence, billing, and machine learning model training. The pipeline processes approximately 50 million events per day from 15 distinct sources, with a total daily data volume of approximately 200 GB before transformation and 80 GB after compression and deduplication. The pipeline follows a modern lakehouse architecture with three layers: the Bronze layer (raw ingestion) stores data exactly as received from source systems in Apache Parquet format on S3 (or the on-premises NFS equivalent), providing a complete audit trail and enabling replay of any transformation. The Silver layer (cleaned and standardized) applies schema validation, data type normalization, deduplication, and PII masking. Data in the Silver layer is stored in a partitioned ClickHouse table optimized for time-range queries, which serves as the primary source for analytical queries. The Gold layer (business-ready aggregates) contains pre-computed metrics, KPIs, and dimensional models that power dashboards and reports. Gold layer tables are materialized views in ClickHouse that refresh on a schedule, providing sub-second query performance for the most common business questions. Data lineage is tracked using Apache Atlas, which captures the relationship between source systems, transformations, and output tables. Every transformation is a versioned Python function with unit tests, and changes to transformation logic go through the same code review process as application code changes.

2. Data Sources and Ingestion

2.1 Real-Time Sources (Kafka)

The primary real-time data sources are Kafka topics published by the ChatAssist microservices. The following topics are consumed by the data pipeline: 'conversation.events' (approximately 8 million events/day) contains conversation lifecycle events including creation, participant changes, and archival. Each event includes the conversation_id, organization_id, event_type, timestamp, and a payload specific to the event type. The schema is versioned using Avro with the Confluent Schema Registry, currently at version 14. 'message.events' (approximately 40 million events/day) contains individual message events — the highest volume topic. Each event includes message_id, conversation_id, sender_type (user, ai, agent), message_content (encrypted at rest), timestamp, and AI-specific metadata (intent classification, confidence score, retrieval sources) for AI-generated messages. The message content is encrypted using AES-256-GCM with a per-organization encryption key stored in HashiCorp Vault. The data pipeline decrypts the content for processing and re-encrypts it with the data warehouse encryption key before writing to the Silver layer. 'feedback.events' (approximately 500,000 events/day) contains user feedback signals including explicit ratings (thumbs up/down, star ratings), implicit signals (message edits, re-asks, escalation to human), and post-conversation CSAT survey responses. These events are critical for AI model evaluation and are consumed by both the data pipeline (for aggregate reporting) and the ML training pipeline (for model fine-tuning dataset construction). 'billing.events' (approximately 2 million

events/day) contains usage metering events used for customer billing. Each event represents a billable action (message sent, API call, file uploaded, etc.) and includes the organization_id, billing_plan_id, action_type, and quantity. These events undergo special handling in the pipeline: they are written to a separate billing-specific Bronze layer with exactly-once delivery guarantees (using Kafka transactions), and the Silver layer transformation includes a reconciliation step that cross-references with the source service's internal counters to detect any discrepancies (learned from INC-2026-003, the data pipeline silent corruption incident described in the Incident Management Case Studies document).

2.2 Batch Sources

Several data sources are ingested in batch mode, either because the source system does not support real-time extraction or because the data changes infrequently: The HR system (TechViet HR Portal): employee data is extracted daily at 01:00 UTC via the HR system's REST API. The extraction uses an incremental strategy based on the last_modified timestamp, fetching only records that changed since the previous extraction. This data feeds into the organizational analytics dashboards showing headcount trends, attrition rates, and department-level metrics. PII fields (personal ID numbers, addresses, bank account numbers) are masked at ingestion using SHA-256 hashing with a salt specific to each field. The Finance system (SAP Business One): financial data including accounts payable, accounts receivable, general ledger entries, and budget allocations is extracted nightly via SAP's OData API. The extraction is a full refresh of the current period (current month) and an incremental update for historical periods. This data is used for financial reporting, cost allocation by department, and vendor payment analysis. Customer CRM (HubSpot): customer account data, deal pipeline, and support ticket information is synced every 6 hours using the HubSpot API. The sync tracks the CRM deal stage for each ChatAssist customer, enabling the sales team to correlate product usage metrics (from the real-time pipeline) with deal progression. External market data: industry benchmarks, competitor pricing data, and market size estimates are manually uploaded by the Strategy team as CSV or Excel files to a designated S3 bucket. The pipeline monitors this bucket and ingests new files within 15 minutes of upload, applying schema validation and format normalization.

3. Transformation Logic

Data transformations are implemented as Python functions using the PySpark framework for batch transformations and Apache Flink (via the PyFlink API) for streaming transformations. All transformations are registered in the transformation catalog (a YAML file in the techviet/data-transforms repository) and are executed by the Airflow orchestrator. The transformation pipeline applies the following steps in order for each data source: Step 1 — Schema validation: incoming records are validated against the registered Avro schema. Records that fail validation are routed to a dead-letter table (bronze_dead_letter) with the validation error message. The dead-letter rate is monitored, and an alert fires if it exceeds 0.1% of total records for any source. Step 2 — Deduplication: records are deduplicated using a composite key specific to each source (e.g., message_id + partition for message events, employee_id + extraction_date for HR data). The deduplication window is configurable per source, defaulting to 24 hours. For billing events, deduplication uses an exactly-once semantic backed by a PostgreSQL table that tracks processed message offsets. Step 3 — PII masking: sensitive fields identified in the data classification matrix are masked according to the field's classification level. Level 1 (Public): no masking. Level 2 (Internal): fields are preserved but access is restricted to authorized roles via ClickHouse row-level security. Level 3 (Confidential): fields are pseudonymized using a deterministic hash that preserves referential integrity across tables. Level 4 (Restricted): fields are tokenized using a reversible tokenization service (for cases where the original value must be recoverable, e.g., for billing disputes), with the token-to-value mapping stored in a separate, access-controlled database. Step 4 — Enrichment: records are enriched with dimensional data from reference tables (e.g., resolving organization_id to organization name, plan, and tier; resolving user_id to user role and department; adding geographic information based on the user's

registered location). Enrichment uses broadcast joins to avoid shuffling the large fact tables. Step 5 — Aggregation: for the Gold layer, aggregation functions compute daily, weekly, and monthly metrics including: message volume by organization, channel, and hour; AI response quality metrics (intent accuracy, retrieval precision, user satisfaction); SLA compliance (response time vs. committed SLA thresholds); and billing summaries by organization and plan tier. The aggregation logic is implemented as SQL materialized views in ClickHouse, refreshed hourly for daily views and nightly for weekly/monthly views.

4. Data Quality Framework

Data quality is monitored using a custom framework built on top of Great Expectations, an open-source data validation library. The framework defines quality expectations for each Silver and Gold layer table, organized into four dimensions: completeness (are all expected records present?), accuracy (do values fall within expected ranges and pass cross-reference checks?), timeliness (was the data delivered within the expected SLA?), and consistency (do related tables agree with each other?). The quality checks run as part of every transformation job (inline checks) and as a separate scheduled job (end-of-day comprehensive check). Inline checks are fail-fast: if a critical quality check fails, the transformation job halts and the on-call data engineer is paged. Non-critical check failures are logged and included in the daily data quality report. The most important quality check is the billing reconciliation check, which runs hourly and compares the message counts in the billing Gold layer with the source-of-truth counts maintained by each microservice. A discrepancy of more than 0.01% triggers an immediate alert. This check was implemented after the INC-2026-003 data pipeline corruption incident and has a 100% detection rate for all simulated failure scenarios in our chaos testing. The data quality dashboard (accessible at analytics.internal.techviet.vn/data-quality) shows real-time quality metrics for all tables, with historical trends and anomaly detection using a rolling 30-day baseline. Department heads receive a weekly email summarizing the quality metrics for their department's data, and the Data Engineering team reviews the overall quality posture in a weekly meeting every Monday at 10:00 Vietnam time.